Foundations of Quantum Computing

–Week 6– Explore- Study

Niel De Beaudrap

This week, we apply what we know about phase estimation to describe what is possibly the most important quantum algorithm discovered so far: Shor's algorithm for factorising integers. The reason why this algorithm is so important, is because the difficulty of factorising large integers is the basis of some of the most widely used cryptographic systems that we know of. The ability to factorise large integers quickly would render a lot of secret information insecure. In this week's Study section you will cover the following topics:

6.1 The RSA cryptosystem

6.2 Factorisation and order finding

6.3 Order finding and phase estimation

6.1 The RSA cryptosystem

As motivation for the integer factorisation problem, we will spend some time considering the Rivest-Shamir-Adleman (RSA) cryptosystem, which is vulnerable (to quantum computers) as a result of Shor's algorithm.

RSA is being phased out as a result of this weakness, but it is not the only cryptosystem which is vulnerable to quantum computers: elliptic curve cryptography is also vulnerable to quantum computers as a result of a very closely related algorithm to solve what is known as the 'discrete logarithm problem'. Investigating cryptographic systems which one should expect to be resistant to attacks by quantum computers, is the subject of the field of **post-quantum cryptography**.

The RSA cryptosystem is a solution to a problem of securely communicating between two different people: 'Alice' and 'Bob'. The problem of doing this 'securely', is to do this in a way that no third person 'Eve' can tell what the meaning of the message is, even if they were to keep a perfect copy. This is done by encoding the message – transforming it in a way that Eve cannot reverse, without some specific knowledge which Bob has.

In the RSA cryptosystem, we consider the message that Alice sends to Bob to be represented as an integer $M > 0$. (It can certainly be represented as a bitstring which can then be interpreted as an integer.) The message has to be shorter than some upper limit N which is part of the scheme set up between Alice and Bob – but a long message can be broken into several smaller messages to allow Alice to send that long message securely. Bob does the following, keeping most of the information secret from Alice:

1. Bob selects a pair of two prime numbers, P and Q – numbers which are greater than 1, and so each can only be evenly divided by 1 and themselves. Bob then sets $N = PQ$ and $\varphi = (P-1)(Q-1)$.

2. Bob chooses a small odd number $e > 1$ at random, subject to the constraint that it is **coprime** to φ: that is, which has no factors in common with φ apart from 1.

3. Bob computes an integer $1 < d < \varphi$, such that $de \equiv 1 \pmod{\varphi}$. Such an integer d is guaranteed to exist for e which is coprime to φ (and computing it is not difficult).

4. To anyone interested in sharing a secret message with Bob, he shares the pair of integers (e, N), which are known as Bob's **public key**. He does not share d or φ with anyone. Indeed, Bob doesn't have any more use for φ after he has computed d. In addition to his public key, which he needs to be able to distribute to other people, he only needs to record the value of d. We refer to the pair of integers (d, N) as Bob's **private key**.

There are technical details which govern how the prime numbers P and Q should be chosen, and how the integer e should be chosen, which are not important to understand the basic principle of RSA. The reason for Bob to perform the calculations is that e and d can be used as the basis for a way to encode and decode messages – where the encoding is difficult to undo, unless either you know d or you are able to recover P and Q from N. It is possible to show that, for any integer $0 < x < N$, that we have:

$$x^{de} \equiv x \pmod{N}. \tag{6.1}$$

This property holds as a direct result of how d is computed from e. (See, for example, Appendix 5 of Ref. [1] for more details.) This means that applying the function:

$$\mathcal{E}(x) = y, \qquad \text{such that } 0 < y < n \text{ and } y \equiv x^e \pmod{N} \tag{6.2}$$

transforms the input x in a way that can be undone by applying the function:

$$\mathcal{D}(y) = z, \qquad \text{such that } 0 < z < n \text{ and } z \equiv y^d \pmod{N}. \tag{6.3}$$

Specifically, $\mathcal{D}(\mathcal{E}(x)) \equiv x^{de} \pmod{N}$, and $x^{de} \equiv x \pmod{N}$, so that $\mathcal{D}(\mathcal{E}(x)) = x$. Then, if Alice has Bob's public key, and a message encoded as an integer $0 < M < n$, she can encrypt it by sending an integer message $C = \mathcal{E}(M)$, and Bob can decrypt the message by computing $M = \mathcal{D}(C)$.

The reason why this should be seen as a secure way of sending messages, is because of the difficulty of decrypting messages. Even if some other person, Eve, knows the public key (e, N), it is not necessarily easy to compute what the value of d should be. In fact, without knowing what the factors P and Q of N are, it is potentially quite difficult if N is a very large number. The technical details of how P, Q, and e are chosen are intended to ensure that Bob does not choose values which are likely to be easy to find by accident. (Factorising large numbers has been considered to be a difficult practical mathematical problem for hundreds of years: there is no known fast way to do it. This does not mean

that it is impossible for us to ever find a fast way of doing it, but that it is a problem which has withstood scrutiny for a long time.) However, anyone who can find out what P and Q are, can then compute φ and determine the value of d just as easily as Bob did originally.

Currently it is believed that the task of factorising large numbers is intractably slow for classical computers (taking an amount of time to run that is exponential in the number of bits needed to state the problem). This has not been **proven** to be the case, but results of that sort are extremely rare for any problem. However, the security of any message sent using RSA relies on there being **no** practical and efficient way to factorise large integers. If Alice and Bob wish for the message to remain secret on a timescale of minutes, this is less of a worry for now – but if Alice and Bob want their secrets to be safe for decades, they must now consider the possibility of scalable quantum computers being built which might compromise their secrets to anyone who might have intercepted and stored a copy of them.

6.2 Factorisation and order finding

We will show how to efficiently solve the problem of integer factorisation, provided that you also have some way of efficiently solving a problem known as **order finding**. (You will not need to understand this in detail: we present it only for the sake of completeness, to spell out how to relate integer factorisation to a problem which we may approach with quantum computers.)

Clearly, this problem is also one which is a difficult problem to solve with a classical computer: if it were not, they would provide an efficient way to factorise large integers. But as we will see, the order finding problem has features which are much more approachable, if you have access to a large quantum computer.

6.2.1 Number theoretic concepts behind the factorisation problem

Suppose that we wish to factorise a large integer N: that is, to find two integers $1 < AB < N$ such that $N = AB$. (The constraint that $1 < A, B < N$, is described as A and B being **non-trivial** factors: that is, factors which are neither 1 nor N itself.) If N is prime, then there are no such integers A and B – but we can efficiently check whether N is prime using an algorithm on a classical computer, with either a randomised or deterministic algorithm (see e.g Ref. [2]). (See the Independent Study material from Week 1 on Randomness and primality tests for more detail.)

Whether or not N is prime, it will have a **prime factorisation**: a sequence of prime numbers $p_1, p_2, \ldots, p_k$ and integer exponents $e_1, e_2, \ldots, e_k > 0$ such that:

$$N \;=\; p_1^{e_1}\, p_2^{e_2} \,\cdots\, p_k^{e_k}. \tag{6.4}$$

If N is prime, then $k = 1$ and $e_1 = 1$, so that $N = p_1^{e_1} = p_1$.) If N is not prime, we say that it is **composite**.

One way that N can be composite, is if it is a non-trivial power of a single prime number: that is, if $N = p_1^{e_1}$ for some $e_1 > 1$. Because $p_1 > 2$, it is not difficult to show that in this case we would have $e_1 < \log_2(N)$. Then, it is feasible to test for every $1 < r < \log_2(N)$ whether $N^{1/r}$ is an integer. (An efficient algorithm to do just this, to test whether N is a perfect power of **any** integer, is presented as Exercise 5.17 of Ref. [1].) We may then suppose that for any integer we wish to factorise, that we check whether it is a perfect power first.

If N is not a perfect power, then in particular $k > 1$. There will then be at least one way to factorise $N = AB$ into two factors, such that A and B are coprime to each other (so that they have no common factors). Conversely, any factorisation of $N = AB$ into coprime factors A and B, will divide the prime factors $p_1, p_2, \ldots, p_k$ into disjoint sets: the primes that are factors of A, and those that are factors of B.

To factorise an integer N, we may consider how to try to partition the prime factors of N to obtain a factorisation. We may simplify this by observing that if N is even, we already know that 2 is a factor, in which case we may factorise $N = AB$ for $A = 2$ (or for A as large a power of 2 as we can find to divide N), and $B = N/A$. So, suppose that N is odd. We then have the following:

Claim 1. *factorSplitting (Splitting the factors of N) Suppose that we have an odd integer N and an integer $1 < u < N - 1$ such that $u^2 \equiv 1 \pmod{N}$. Then $u - 1$ and $u + 1$ both have a factors in common with N, and in fact $N = AB$ for $A = \gcd(u - 1, N)$ and $B = \gcd(u + 1, N)$. (If N has more than one prime factor, such an integer u is guaranteed to exist.)*

(Here, $\gcd(x, y)$ is the **greatest common divisor** of two integers x and y, which may be computed efficiently from x and y in $\mathcal{O}(n^3)$ time, where n is the number of bits required to represent x and y in binary.) This is actually easy to show, making use of a couple of results of number theory.

- If $u^2 \equiv 1 \pmod{N}$, then $(u + 1)(u - 1) = u^2 - 1 \equiv 0 \pmod{N}$. It is possible to show that this means that $(u + 1)(u - 1) \equiv 0 \pmod{{p_j}^{e_j}}$ for each prime factor p_j of N. As N is not even, at most one of $u - 1$ and $u + 1$ can even be divided by any given prime factor p_j. So this implies that for each p_j, either $u - 1 \equiv 0 \pmod{{p_j}^{e_j}}$ or $u + 1 \equiv 0 \pmod{{p_j}^{e_j}}$. That is, we have $u \equiv \pm 1 \pmod{{p_j}^{e_j}}$ for each p_j.

- There is a well-known result of number theory – which we call Sun Tsu's Remainder Theorem (it is commonly known by a similar name) – which as a special case guarantees that for $0 \leqslant u < N$, $u = 1$ is the unique integer for which $u \equiv +1 \pmod{{p_j}^{e_j}}$ for all p_j; and that $u = N - 1$ is the unique integer for which $u \equiv -1 \pmod{{p_j}^{e_j}}$ for all p_j. Because $1 < u < N - 1$ by hypothesis, we then know that there are some primes p_j for which $u \equiv +1 \pmod{{p_j}^{e_j}}$, and some primes p_j for which $u \equiv -1 \pmod{{p_j}^{e_j}}$.

- Thus, we may partition the prime factors p_j of N into two separate non-empty collections: those for which ${p_j}^{e_j}$ divides $u-1$, and those for which ${p_j}^{e_j}$ divides $u+1$. Then $A = \gcd(u-1, N)$ is the product of the ${p_j}^{e_j}$ which divide $x-1$, and $B = \gcd(u+1, N)$ is the product of the ${p_j}^{e_j}$ which divide $x+1$.

This allows us to factorise N, provided that we can find such a convenient integer x.

If N has more than one prime factor p_j , there are guaranteed to be at least two integers u with this property, and the more of them there are, the more distinct prime factors $p_1, p_2, \ldots, p_k$ that N has. For each way that we can partition the prime factors p_j into two non-empty sets S and T, we can find an integer u such that:

$$\begin{aligned} u &\equiv -1 \pmod{{p_j}^{e_j}}, \quad \text{if } p_j \in S, \\ u &\equiv +1 \pmod{{p_j}^{e_j}}, \quad \text{if } p_j \in T. \end{aligned} \tag{6.5}$$

You do not have to understand why this holds: this is again a special case of Sun Tsu's Remainder Theorem. The problem is that we cannot easily find such a value of u, if we don't know the prime power factors ${p_j}^{e_j}$. (If we did know them, we would already know how to factorise N.)

The abundance of numbers u for which $u^2 \equiv 1 \pmod{N}$ **may be surprising** if (as we recommend you do) you are trying to think of modular arithmetic as being **mostly** like normal arithmetic. If you are mainly used to thinking about real and complex numbers, you may normally expect a relation such as $u^2 = 1$ to imply that $u = \pm 1$. But with modular arithmetic, properties such as this hold because $\mathbb{R}$ and $\mathbb{C}$ have multiplicative inverses for all of their non-zero elements. However, for N composite, there are integers $1 < a, b < N$ for which $ab \equiv 0 \pmod{N}$, meaning that you cannot undo the effect of multiplying by either a or by b when doing arithmetic modulo N: they do not have multiplicative inverses. The property of an integer $1 < a < N$ having an inverse 'modulo N' is not very rare, but it is note-worthy: we say then that a is a **unit** modulo N.

6.2.2 Introducing: the order finding problem

There is one more property of modular arithmetic which you may be surprised by, but which is relevant to this problem. For any integer $1 < a < N$ consider the powers of a modulo N: that is, the numbers:

$$v_0 = a^0, \qquad v_1 = a^1, \qquad v_2 \equiv a^2 \pmod{N}, \qquad v_3 \equiv a^3 \pmod{N}, \tag{6.6}$$

and so forth, taking $1 < v_j < N$ in each case. Because there are only so many different integers $0 \leqslant x < N$, eventually they have to repeat: there will be some $v_s \equiv a^s \pmod{N}$ and $v_t \equiv a^t \pmod{N}$ for $t < s$, such that $v_s = v_t$. Then, we can have $a^t \equiv a^s \pmod{N}$ for different integer exponents $t < s$.

If a is a unit modulo N, we can undo multiplication modulo a: there is some $1 < b < N$ such that $ab \equiv 1 \pmod{N}$. We write $b \equiv a^{-1} \pmod{N}$ in this case, and $b^t \equiv a^{-t} \pmod{N}$ more generally. We then have:

$$a^{s-t} \;\equiv\; a^s\, a^{-t} \;\equiv\; a^t\, a^{-t} \;\equiv\; 1 \pmod{N}. \tag{6.7}$$

That is, the cycle repeats with $v_t = v_0 = 1$. We call the smallest integer t for which $a^t \equiv 1 \pmod{N}$, the **order** of a modulo N.

When we consider a number $1 < x < N$ such that $x^2 \equiv 1 \pmod{N}$, this basically amounts to some number which is **exactly** half-way through the sequence of powers a^0, a^1, a^2, ... modulo N, before they repeat for the first time, for some unit a which happens to have even order. (Not all units necessarily have even order: for instance, for $N = 13$ and $a = 3$, we have $a^1 = 3$, $a^2 = 9$, and $a^3 = 27 \equiv 1 \pmod{N}$, so that a has order three.) Therefore to find such an x, it would be enough to find some unit a modulo N which has even order, to **find out** the order t of a, and to compute $x \equiv a^{t/2} \pmod{N}$. Because $t/2$ is less than the order of a, we would be guaranteed that $x \not\equiv 1 \pmod{N}$. Two questions then, are:

- how to find a unit a of even order t, such that $x \equiv a^{t/2} \not\equiv -1 \pmod{N}$, so that we can apply Claim 1?
- how to find the order t of a, which we would need to do to check whether t is even, and to compute $x \equiv a^{t/2} \pmod{N}$?

It is possible to show (see Theorem 5.3 of Ref[1]) that we can take a unit of N completely at random and the probability with which it satisfies the first constraint is reasonably high: at least $1 - 2^{-k}$, where $k \geqslant 2$ is the number of prime factors of N. The probability with which a randomly chosen $1 < a < N$ is a unit in the first place is also always large enough that we can find one without too much trouble – but for the factorisation problem, we would be perfectly happy to immediately stumble upon $1 < a < N$ which instead has factors in common with N, as we could then compute a factor $r = \gcd(a, N)$. The only remaining problem then is how to find the order of a.

This is what we mean by the **order finding problem**: given two integers N and $1 < a < N$, where a is coprime to N, to find the order of a modulo N. Again: clearly, finding the order of a unit a modulo N is a problem which we do not know how to solve efficiently using conventional computers. But if we consider how we would realise a subroutine to multiply an input $1 < x < N$ modulo N by a unit a, we can see that this is related to the question of phase estimation.

6.2.3 Summary: reducing from factorisation to order finding

We summarise this section by describing how we may factorise an integer N, if we have some way to find the order of a unit a modulo N. The following is an algorithm which takes a composite integer N, and (if it succeeds) produces a pair of integers (A, B) such that $N = AB$.

As the procedure that follows is almost entirely classical, with one step that uses a quantum subroutine, we write it out as a sequence of steps rather than pseudocode. It relies on a single step which cannot be done efficiently by any known classical means: finding the order of an integer a, modulo N. Let $0 < p < 1$ be some upper bound on the desired probability of failure:

1. If N is even, return $(2,\ N/2)$ and stop. Otherwise, continue.

2. For each $1 < r < \log_2(N)$, determine whether $N^{1/r}$ is an integer. If so, return $(N^{1/r},\ N^{(r-1)/r})$ and stop. Otherwise, continue.

3. Repeat the following at most $\lceil \frac{1}{2}\log_2(1/p) \rceil$ times.

   (a) Choose $1 < a < N$ uniformly at random. If $g = \gcd(a, N) > 1$, return $(g, N/g)$ and stop. Otherwise, continue.

   (b) **Find t, the order of a modulo N.** If t is odd, we retry Step 3. Otherwise, we continue.

   (c) Let $u \equiv a^{t/2} \pmod{N}$. If $u \equiv -1 \pmod{N}$, we retry Step 3. Otherwise, we proceed to Step 4.

   If the loop finishes without proceeding to Step 4, stop and indicate that the algorithm has failed.

4. Compute $A = \gcd(u - 1, N)$ and $B = \gcd(u + 1, N)$, and return (A, B).

If the algorithm proceeds past Step 1, N is odd; if it proceeds past Step 2, it has $k \geqslant 2$ prime factors. The only step with some probability of failure is Step 3, where each iteration has a probability of at most $2^{-k} < \frac{1}{4}$ of forcing a retry (as $k \geqslant 2$ if the algorithm gets to Step 3). Then, the probability with which the algorithm fails is at most $(\frac{1}{4})^{\lceil \log_2(1/p)\,/\,2 \rceil} < 2^{\log_2(p)} = p$. If the algorithm makes it to Step 4, it produces a factorisation.

6.3 Order finding and phase estimation

We have described before how integer factorisation plays an important role in the RSA cryptosystem and how it can be 'reduced' to the order finding problem – that is, how it is possible to solve integer factorisation for N, if you can solve the order finding problem for a unit a modulo N.

As the order-finding problem is about exponentiating (or more generally, multiplication by) an integer a modulo N, it is worth pausing to think of what that would look like with a quantum computer. We would generally represent this by a unitary transformation U_a, acting on n qubits for n large enough to represent the integers $0 \leqslant x \leqslant N - 1$, which performs the transformation:

$$|x\rangle \xmapsto{U_a} \begin{cases} |ax \mod N\rangle\,, & \text{if } x < N; \\ |x\rangle\,, & \text{if } x \geqslant N. \end{cases} \tag{6.8}$$

On the right hand side, the expression $|ax \mod N\rangle$ represents a state $|y\rangle$, for an integer $0 \leqslant y < N$ such that $y \equiv ax \pmod{N}$. The order of a, is precisely the smallest integer $t > 0$ for which $U_a^t = I$. This means that all of the eigenvalues $\mathrm{e}^{i\theta}$ of U_a, also satisfy $(\mathrm{e}^{i\theta})^t = 1$ – so that θ is an integer multiple of $2\pi/t$. If we can get precise enough information about the eigenvalues of U_a, this then may provide a way to solve the order finding

problem, and thus integer factorisation as well.

Next, we consider how we can approach this idea, as a lead-up to describing Shor's algorithm – a procedure which uses phase estimation on random eigenvectors of an operator U_a, to solve the order-finding problem.

6.3.1 The quantum Fourier transform

In the Week 4 Study materials (Section 4.2.5), we considered a procedure which produced states of the form:

$$|f_{e^{i\theta}}\rangle \;=\; \tfrac{1}{\sqrt{2^n}}\Big(|0\rangle + e^{i\theta}\,|1\rangle + e^{2i\theta}\,|2\rangle + \cdots + e^{(2^n-1)i\theta}\,|2^n{-}1\rangle\Big) \tag{6.9}$$

for some angle θ, and where n is the number of qubits storing the state (with the basis states represented in binary). In the cases we considered there, we had $n = 2$ or $n = 3$, and the basis states $|k\rangle$ were just represented by two- or three-bit strings. In the case that $\theta = \frac{2\pi}{2^n}$, we described unitary transformations – the **quantum Fourier transform** (QFT), modulo 4 or 8 (for $n = 2$ or $n = 3$ respectively – to transform these states to computational basis states. We now consider how to generalise this to any $n \geqslant 1$.

Consider the case that θ is exactly some $1/2^n$ fraction of a full revolution (so that $\theta = k\pi/2^{n-1}$ for some integer $0 \leqslant k < 2^n$). Let $\omega = e^{2\pi i/2^n}$. For each of the 2^n different values of k, the states $|f_a\rangle = |f_{\omega^k}\rangle$ that result are actually orthogonal to one another. For any integers $0 \leqslant h, k < 2^n$, we have:

$$\begin{aligned}\langle f_{\omega^h}\,|f_{\omega^k}\rangle \;&=\; \tfrac{1}{2^n}\Big(1 \,+\, \omega^{-h}\cdot\omega^{k} \,+\, \omega^{-2h}\cdot\omega^{2k} \,+\, \cdots \omega^{-(2^n-1)h}\cdot\omega^{(2^n-1)k}\Big)\\ &=\; \tfrac{1}{2^n}\Big(1 \,+\, (\omega^{k-h}) \,+\, (\omega^{(k-h)})^2 \,+\, \cdots \,+\, (\omega^{(k-h)})^{(2^n-1)}\Big).\end{aligned} \tag{6.10}$$

The sum on the bottom line is a sum of powers of the complex number $\omega^{(k-h)} = e^{2\pi i(k-h)/2^n}$. If $k = h$, then each of the terms in the sum is simply 1, so that $\langle f_{\omega^h}\,|f_{\omega^k}\rangle = \frac{1}{2^n}\cdot 2^n = 1$, as we would expect. If $k \neq h$, then the terms in the sum represent a sum of various N^{th} roots of unity, which perfectly cancel each other out to give 0. This can easily be demonstrated by a geometric series – but it can also be understood geometrically, as in Figure 6.1, as a result of the sum being the result of adding together a collection of perfectly evenly distributed points on the unit circle in the complex plane. (If $k - h$ happens to be an odd number, these will be all of the N^{th} roots of unity, for $N = 2^n$. If $k - h$ is even, let 2^m be the largest power of 2 which evenly divides $k - h$: then the terms of the sum are all of the N^{th} roots of unity for $N = 2^{n-m}$, and where each term occurs 2^m times in the sum.)

Because the states $|f_{\omega^k}\rangle$ are all orthogonal unit vectors, we may consider a unitary transformation which has these state-vectors for its columns:

$$F_{2^n} \;=\; \frac{1}{\sqrt{2^n}}\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{-2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-1} & \omega^{-2} & \cdots & \omega \end{bmatrix}. \tag{6.11}$$

Figure 6.1: Roots of unity. These two diagrams show the set of the N^{th} roots of unity as points in the complex plane, for $N = 8$ and $N = 16$. Because they are uniformly distributed around the unit circle in the complex plane, their sum is the centre of that circle – which is zero.

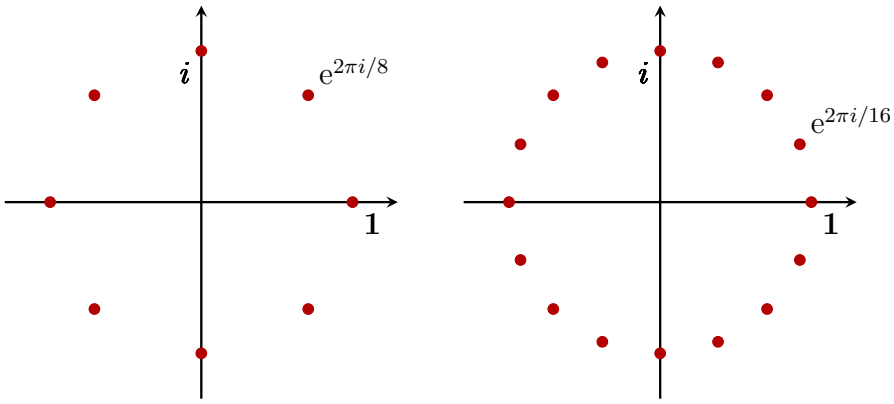

Figure 6.1 Long description: Two sets of horizontal and vertical axes are shown. For both sets of axes, there is a point along the positive horizontal axis labeled by 1, and a point along the positive vertical axis labeled by i, indicating that each of these diagrams represent the complex number plane. On the first set of axes, there are eight dark red dots equally spaced on the unit circle (including the points labeled by 1 and i). The first node anticlockwise from the node marked 1, is labeled by the complex number $e^{2\pi i/8}$. On the second set of axes, there are sixteen dark red dots equally spaced on the unit circle (again including the points labeled by 1 and i). The first node anticlockwise from the node marked 1, is labeled by the complex number $e^{2\pi i/16}$.

This is the **quantum Fourier transform** on n qubits (or the quantum Fourier transform modulo 2^n), generalising the operation described in Eqn. (4.33) of the Week 4 Study materials. We often refer to this operation as the QFT on n qubits, for short. This operation performs the transformation $F_{2^n}|k\rangle = |f_{\omega^k}\rangle$, for computational basis states $|k\rangle$ taken to represent integers $0 \leqslant k < 2^n$. In practise, however, it is the inverse operation, $F_{2^n}^\dagger$, which is often more useful in algorithms:

$$F_{2^n}^\dagger = \frac{1}{\sqrt{2^n}} \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \cdots & \omega \\ 1 & \omega^{-2} & \omega^{-4} & \cdots & \omega^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega & \omega^2 & \cdots & \omega^{-1} \end{bmatrix}. \tag{6.12}$$

This operation is simply called the 'inverse quantum Fourier transform' (on n qubits), or the inverse QFT. It is useful to perform the transformation:

$$|f_{\omega^k}\rangle \xmapsto{F_{2^n}^\dagger} |k\rangle\,. \tag{6.13}$$

This then transforms the superposition state $|f_{\omega^k}\rangle = \frac{1}{\sqrt{2^n}}\big(|0\rangle + \omega^k|1\rangle + \omega^{2k}|2\rangle + \cdots\big)$ with information encoded in the relative phases, to the state $|k\rangle$ where that same information is represented directly in the computational basis.

6.3.2 Realising the QFT in QAAL

Although the linear maps presented in Eqns. (6.11) and (6.12) are described succinctly enough, this does not give an explicit description of how to realise them using practical operations (for example, as a QAAL subroutine). To do this we will consider how we may represent the states $|f_{\omega^k}\rangle$ in a slightly different way.

We do not need to memorise the following, it is presented only to demonstrate how a QAAL procedure may be derived. (For an alternative presentation of the following line of reasoning, see Section 5.1 of Ref. [1]: we also mirror that presentation a little more closely in the next section.) Notice that:

$$\begin{aligned}
|f_{\omega^h}\rangle &= \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} (\mathrm{e}^{2\pi i h/2^n})^k \, |k\rangle \\
&= \frac{1}{\sqrt{2^n}} \sum_{k_{n-1}} \sum_{\cdots} \cdots \sum_{k_1 k_0 \in \{0,1\}^n} \mathrm{e}^{2\pi i \cdot h(2^{n-1}k_{n-1} + 2^{n-2}k_{n-2} + \cdots + 2k_1 + k_0)/2^n} \, |k_{n-1}, k_{n-2}, \ldots, k_1, k_0\rangle \\
&= \left(\frac{|0\rangle + \mathrm{e}^{2\pi i h/2} \, |1\rangle}{\sqrt{2}} \right) \otimes \left(\frac{|0\rangle + \mathrm{e}^{2\pi i h/4} \, |1\rangle}{\sqrt{2}} \right) \otimes \cdots \otimes \left(\frac{|0\rangle + \mathrm{e}^{2\pi i h/2^n} \, |1\rangle}{\sqrt{2}} \right). \qquad (6.14)
\end{aligned}$$

That is: the states $|f_{\omega^h}\rangle$ aren't even entangled. What is more, by considering the way that $0 \leqslant h < 2^n$ is represented by binary strings, we can describe the way that each of the single-qubit factors $\frac{1}{\sqrt{2}}\big(|0\rangle + \mathrm{e}^{2\pi i h/2^r} \, |1\rangle\big)$ often depends on some limited number of the input bits $h_{n-1}h_{n-2}\cdots h_1 h_0 \in \{0,1\}^n$. We may observe that:

$$\begin{aligned}
\mathrm{e}^{2\pi i h/2^r} &= \mathrm{e}^{2\pi i (2^{n-1}h_{n-1} + 2^{n-2}h_{n-2} + \cdots + 2h_1 + h_0)/2^r} \\
&= \mathrm{e}^{2\pi i \cdot 2^{n-r-1}h_{n-1}} \, \mathrm{e}^{2\pi i \cdot 2^{n-r-2}h_{n-2}} \, \cdots \, \mathrm{e}^{2\pi i \cdot 2^{1-r}h_1} \mathrm{e}^{2\pi i \cdot 2^{-r}h_0} \, . \qquad (6.15)
\end{aligned}$$

As $\mathrm{e}^{2\pi i N} = 1$ for any integer N, we have $\mathrm{e}^{2\pi i \cdot 2^{s-r}h_s} = 1$ whenever $s \geqslant r$. So we can simplify the preceding equation to:

$$\begin{aligned}
\mathrm{e}^{2\pi i h/2^r} &= \mathrm{e}^{2\pi i \cdot 2^{-1}h_{r-1}} \, \mathrm{e}^{2\pi i \cdot 2^{-2}h_{r-2}} \, \cdots \, \mathrm{e}^{2\pi i \cdot 2^{-r}h_0} \\
&= \mathrm{e}^{2\pi i \cdot h_{r-1}/2} \, \mathrm{e}^{2\pi i \cdot h_{r-2}/4} \, \cdots \, \mathrm{e}^{2\pi i \cdot h_0/2^r} \, . \qquad (6.16)
\end{aligned}$$

From this, it follows that:

$$\left(\frac{|0\rangle + \mathrm{e}^{2\pi i h/2^r} \, |1\rangle}{\sqrt{2}} \right) = \left(\frac{|0\rangle + \mathrm{e}^{2\pi i \cdot h_{r-1}/2} \, \mathrm{e}^{2\pi i \cdot h_{r-2}/4} \, \cdots \, \mathrm{e}^{2\pi i \cdot h_0/2^r} \, |1\rangle}{\sqrt{2}} \right). \qquad (6.17)$$

This provides a description of the state of the r^{th} qubit of $|f_{\omega^h}\rangle$, in terms of the 'final' r bits of h (where we represent the integer h in terms of the binary string $h_{n-1}h_{n-2}\cdots h_1 h_0 \in \{0,1\}^n$, with h_{n-1} coming first and h_0 coming last).

This can be used as the basis of an approach to realise the QFT, using a QAAL subroutine. The main idea is to use a loop to prepare each of the qubits of $|f_{\omega^h}\rangle$ in turn, where the

Figure 6.2: The QAAL subroutine QFT[n]. This subroutine accepts an integer meta-parameter n, and performs the quantum Fourier transform on n qubits, informed by the preceding sketch of how to describe $|f_{\omega^h}\rangle = F_{2^n}|h\rangle$ as a product state. This relies on a subroutine ReverseQubits[n], which reverses the order of the qubits in a register (as defined in the Prepare notes for Week 4).

```
define QFT [ n ] q :
   # Perform the QFT on a register q of n qubits
   metaparameter integer n
   operand register q: n qubits     # depends on metaparameter n

   # loop iterators for the qubits to be acted on
   !integer r := n
   !integer s := 0

   @repeat:
      r := r-1                  # proceed to the next lowest qubit
      H q[r]                    # perform a Hadamard on qubit r
      !jump_if_zero r, @done    # break the loop after the last op'n
      s := r                    # otherwise, begin the inner loop
      @cphase_loop:
         s := s - 1             # consider the next lowest qubit
         CRz( π / 2^(r-s) ) q[r], q[s]
         !jump_if_nonzero s, @cphase_loop
      !jump @repeat
   @done:
   ReverseQubits[n] q           # reverse the order of q
   stop
```

relative phases of these qubits are set by controlled-Rz($\pi/2^r$) operations. The notable exception for each qubit $1 \leqslant r \leqslant n$ is the phase contribution of $e^{2\pi i h_{r-1}/2} = (-1)^{r-1}$ to the relative phase: this may be realised through a Hadamard operation, using the fact that $\mathtt{H}|h_{r-1}\rangle = \frac{1}{\sqrt{2}}\big(|0\rangle + (-1)^{h_{r-1}}|1\rangle\big)$.

Suppose that we take as input, a qubit register q of length n, initially in a computational basis state $|h\rangle$. We may prepare the final qubit of $F_{2^n}|h\rangle$ by taking the qubit q[$n-1$] storing the state $|h_{n-1}\rangle$, performing H operation on it, and interacting it with the qubits q[$n-2$], ..., q[1], q[0] with appropriate CRz operations. We may then proceed to prepare the second-to-last qubit of $F_{2^n}|h\rangle$ by performing H on q[$n-2$], and interacting that qubit with q[$n-3$], ..., q[1], q[0], again with appropriate CRz operations. We proceed until we have performed H on q[0] and then reverse the order of the qubits of q so that the qubits of $F_{2^n}|h\rangle$ are presented in the correct order.

A subroutine QFT[n] along these lines is defined in Figure 6.2. (We may similarly define a procedure invQFT[n], to realise the inverse QFT, simply by reversing the order of the operations in QFT[n] and replacing each operation with its inverse. This is presented in

Figure 6.3.)

Figure 6.3: The `QAAL` subroutine `invQFT[n]`. This subroutine performs the inverse of `QFT[n]`, and in particular performs the inverse operations to `QFT[n]` and in the reverse order. This relies on a subroutine `ReverseQubits[n]`, which reverses the order of the qubits in a register (as defined in subsection **(d) Scalable subroutines and classical parameters** of the Prepare notes for Week 4).

```
define invQFT [ n ] q :
   # Perform the QFT on a register q of n qubits
   metaparameter integer n
   operand register q: n qubits     # depends on metaparameter n

   # loop iterators for the qubits to be acted on
   !integer r := 0
   !integer s := 0

   ReverseQubits[n] q              # reverse the order of q

   @repeat:
      s := 0                       # begin the inner loop
      @cphase_loop:
         !jump_if_geq s, r, @end_cphase_loop
         CRz( -π / 2^(r-s) ) q[r], q[s]
         s := s + 1
         !jump @cphase_loop
      @end_cphase_loop:
      H q[r]                       # perform a Hadamard on qubit r
      r := r + 1                   # increment qubit index r
      !jump_if_less r, n, @repeat  # if r < n, continue

   stop
```

6.3.3 Using the inverse QFT for phase estimation

So far, we have motivated the QFT (and the inverse QFT) simply by considering the idea that we might want to consider the states $|f_{\omega^h}\rangle$, as generalising states that we saw in the Week 4 Study materials (and transform between these states and the computational basis).

The reason why we considered such states in the first place was as an approach to getting information about the eigenvalues of an operation `U` by means of phase kicks. In Week 4, we considered the case of $n = 2$ and $n = 3$, where `U` has eigenvalues which are fourth roots of unity (complex phases of the form $\mathrm{e}^{\pi ih/2} = \mathrm{e}^{2\pi ih/4}$ for an integer h), of eighth roots of unity (complex phases of the form $\mathrm{e}^{\pi ih/4} = \mathrm{e}^{2\pi ih/8}$ for an integer h). Generalising this, suppose that we have a unitary operation `U` on n qubits and this has eigenvalues of the form ω^k, where $\omega = \mathrm{e}^{2\pi ik/2^n}$ is a 2^n-th root of unity. If we have a state $|\phi\rangle$ which

is an eigenvector of `U`, we can find its eigenvalue ω^h by performing a phase-kick using `CU` subroutines, to produce a state-vector $|f_{\omega^h}\rangle$, and then mapping this to $|h\rangle$ using the inverse QFT. (To realise `CU`, we generally have to know how `U` is implemented, but for practical problems this is not an obstacle.)

What can we do, if `U` doesn't have such specific eigenvalues? Almost all unitary operations will have eigenvalues which are not a 2^n-th root of unity, for any n: instead it will have eigenvalues of the form $a = \mathrm{e}^{i\theta} = \mathrm{e}^{2\pi i x}$, where $0 \leqslant x < 1$ may be a more general rational or even irrational number. For an eigenvector $|\phi\rangle$ with eigenvalue a, we can still perform phase-kicks into an n-qubit register: this does not rely in any way on what the specific eigenvalues of `U` are. This yields a more general state of the form described in Eqn. (6.9),

$$\begin{aligned}
|f_a\rangle &= \tfrac{1}{\sqrt{2^n}}\Big(|0\rangle + a\,|1\rangle + a^2\,|2\rangle + \cdots + a^{2^n-1}\,|2^n{-}1\rangle\Big) \\
&= \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} (\mathrm{e}^{2\pi i x})^k\,|k\rangle \\
&= \frac{1}{\sqrt{2^n}} \sum_{k_{n-1}} \sum \cdots \sum_{k_1 k_0 \in \{0,1\}^n} \mathrm{e}^{2\pi i \cdot x(2^{n-1}k_{n-1} + 2^{n-2}k_{n-2} + \cdots + 2k_1 + k_0)}\,|k_{n-1}, k_{n-2}, \ldots, k_1\,, k_0\rangle \\
&= \left(\frac{|0\rangle + \mathrm{e}^{\pi i \cdot 2^n x}\,|1\rangle}{\sqrt{2}}\right) \otimes \left(\frac{|0\rangle + \mathrm{e}^{\pi i \cdot 2^{n-1} x}\,|1\rangle}{\sqrt{2}}\right) \otimes \cdots \otimes \left(\frac{|0\rangle + \mathrm{e}^{2\pi i x}\,|1\rangle}{\sqrt{2}}\right). \qquad (6.18)
\end{aligned}$$

If $0 \leqslant x < 1$ is closely enough approximated by some fraction of the form $h/2^n$, this state will be very closely aligned to $|f_{\omega^h}\rangle$, in which case applying $F_{2^n}^\dagger$ will produce a state very close to $|h\rangle$. While we do not obtain perfect information about the value of x (and therefore about the eigenvalue a), this might provide an **estimate** for x up to n bits. This is more generally what 'eigenvalue estimation' consists of. **Note:** Whetheror not the estimate is useful, depends in part on whether n is large enough that we can obtain a good estimate of x.

A note: an important detail that we would wish to consider, in a more complete treatment of phase estimation, is the probability with which we would get **bad** estimates $h/2^n$ for x. It is possible to show (see Eqn. (5.35) of Ref. [1]) that we can limit the probability p of failing to get an inaccurate estimate of x up to $\tilde{n}$ bits, by using a register of $n = \tilde{n} + 1 + \lceil \log_2(1/p) \rceil$ bits to perform the phase kick. (Note that for $p = 0$, this quantity is undefined: but for every additional factor of 2 by which we might wish to reduce the probability of an error, we only need to increase the value of n by 1.)

In Figure 6.4, we present a `QAAL` subroutine to perform phase estimation for some eigenvector of a unitary transformation `U` on m qubits, where the eigenvector is stored in some m-qubit register `t`. This procedure supposes that there is some procedure to realise powers of `CU`, called `CU_Pow(k)`, where `k` is the power to which we wish to raise the operation `CU`. We use multiple applications of `CU_Pow(2**j)` for various values of `j`, to perform a phase-kick into a control register `q`, whose size can be controlled by a meta-parameter.

We then apply the inverse QFT on `q`, and measure it in the computational basis to produce a classical bitstring in a register `x`.

Figure 6.4: A `QAAL` subroutine `Phase_Estimation_U[n]`. This subroutine performs phase estimation for an eigenvector of a unitary `U` on m qubits, to a configurable number of bits of precision. This relies on a subroutine `CU_Pow(k)`, where `CU_Pow(2**j)` performs the 2^j-th power of a controlled-`U` operation, with a single qubit as control.

```
define Phase_Estimation_U [ n ] t -> x :
   # A template procedure to do phase estimation,
   # up to n bits precision, for some fixed m-qubit unitary U
   metaparameter integer n     # number of bits of precision

   operand register x: n bits
   # register for result, representing a fraction between 0 and 1

   operand register t: m qubits
   # stores an eigenvector of some fixed m-qubit unitary U
   # (note that m is fixed and is a property of U)

   register q: n qubits
   # register for the phase-kick (most significant bit is q[n-1])

   !integer j := n             # loop iterator for powers of CU

   H q                         # prepare q in uniform superposition
   @repeat :
      j := j - 1               # reduce exponent of phase kick
      CU_Pow(2**j) q[j], t     # perform a phase kick
      !jump_if_geq j, 0, @repeat

   invQFT[n] q                 # perform inverse QFT
   Mz q -> x                   # measure the result
   stop
```

Note that we can always realise `CU_Pow(k)` just with repeated applications of `CU`. But because we invoke `CU_Pow(2**j)` for values of `j` ranging from 1 up to n, doing it in this way would involve exponentially many applications of `CU`. Therefore, for any given unitary `U`, a more efficient approach to realising `CU_Pow(2**j)` would be preferable. This is not something that we can generically do for an arbitrary operation `U`, even if we knew a way to implement the operation `U` itself. However, for some operations `U` – for example, certain simple permutation operations – we can realise the operations `CU_Pow(2**j)` in particular, by pre-computing what the 2^j-th powers of `U` are.

6.3.4 Modular exponentiation for order finding

The subroutine of Figure 6.4 provides us with a way of finding out what the eigenvalue is of a particular eigenvector of an m-qubit unitary `U`, but relies on a fast way to perform powers $\texttt{CU}^{2^j}$ of a controlled-`U` operation. We don't have access to such a procedure

in general. But for the order finding problem, we have enough information about the operator $\texttt{U} = U_a$, that this can be done without much difficulty. Note that:

$$|x\rangle \xmapsto{U_a^r} |a^r x \bmod N\rangle = |v_r\, x \bmod N\rangle\,, \tag{6.19}$$

which is just multiplication by a different integer $v_r \equiv a^r \pmod N$. Given a unit a, we can simply compute each of the values $v_0 = 1$, $v_1 = a$, $v_2 \equiv a^2 \pmod N$, and so forth up to $v_{n-1} \equiv a^{2^{n-1}} \pmod N$ and then implement different unitary operations which realise the unitary transformation $\mathrm{C}U_{v_j} = (\mathrm{C}U_a)^{2^j}$.

In principle, one would generally choose to evaluate the integers $v_j \equiv a^{2^j} \pmod N$ and define subroutines to realise the transformation $\mathrm{C}U_{v_j}$ as efficiently as possible. For the sake of simplicity, we instead suppose that we have access to a procedure `CMul(v)`, which for a unit v modulo N, realises the transformation $\mathrm{C}U_v$ on a single-qubit control and an m-qubit target register:

$$|c, x\rangle \xmapsto{\texttt{CMul(v)}} \begin{cases} |0, x\rangle\,, & \text{if } c = 0 \text{ or } x \geqslant N; \\ |1, vx \bmod N\rangle\,, & \text{if } c = 1 \text{ and } x < N. \end{cases} \tag{6.20}$$

Then for the case where $\texttt{U} = U_a$, we may realise `CU_Pow(2**j)` by the subroutine $\texttt{CMul}(v_j)$ for $v_j \equiv a^{2^j} \pmod N$. We use this to define a subroutine `Phase_Estimation_Mul[n]` (shown in Figure 6.5), which refines the procedure `Phase_Estimation_U[n]` to make explicit how the phase kicks are to be performed for the application of order finding.

6.3.5 Sampling from useful eigenvectors of U_a

There is a final detail to be resolved: in the use of phase estimation to find the order of U_a, we have not described how to obtain an eigenvector $|f_{\omega^k}\rangle$ in the first place to allow us to perform a phase kick. This is a rather important detail, as the entire premise of phase estimation rests on the premise that we have a specific phase to estimate. But even for the specific m-qubit unitary U_a in the order-finding problem, such eigenvectors will be difficult to construct without knowing the order, t, in advance.

While we cannot easily construct eigenvectors for U_a, we do know what form they have. We won't describe all of the eigenvectors of U_a in detail, but we can describe a few which are useful.

One of the eigenvectors of U_a will simply be the uniform superposition of states of the form $|a^j \bmod N\rangle$ for $0 \leqslant j < t$:

$$|\phi_1\rangle = \tfrac{1}{\sqrt{t}}\Big(|1\rangle + |a\rangle + |a^2 \bmod N\rangle + \cdots + |a^{t-1} \bmod N\rangle\Big). \tag{6.21}$$

This is easy to see: the effect of U_a is just to shift each of the terms in the sum by one position to another of the terms.

Figure 6.5: A QAAL subroutine Phase_Estimation_Mul[n]. This subroutine performs phase estimation for an eigenvector of a unitary U on m qubits, to a configurable number of bits of precision. This relies on a subroutine CMul(v) which performs a controlled-U_v operation with a single qubit as control, where $U_v |x\rangle = |vx \bmod N\rangle$ for $x < N$ and $U_v |x\rangle = |x\rangle$ for $x \geqslant N$.

```
define Phase_Estimation_Mul [ n ] ( a, N ) t -> x :
   # A procedure to do phase estimation, to n bits precision,
   # for multiplication by a unit a modulo of an m-bit integer N
   metaparameter integer n     # number of bits of precision

   parameter integer N         # an m-bit integer
   parameter integer a         # integer which is a unit modulo N

   operand register x: n bits
   # register for result, representing a fraction between 0 and 1

   operand register t: m qubits
   # stores an eigenvector of some fixed m-qubit unitary U
   # (note that m is fixed and is a property of U)

   register q: n qubits
   # register for the phase-kick (most significant bit is q[n-1])

   !integer j := 0             # loop iterator for powers of CU
   !array v: n integers        # array to store integers a^(2^j) mod N

   v[0] := a                   # initialise v[0]
   @mod_expon_loop:            # loop to initialise other v[j]:
      v[j+1] := v[j]**2 mod N  # evaluate by squaring modulo N
      j := j+1
      !jump_if_less j, n, @mod_expon_loop

   H q                         # prepare q in uniform superposition
   @repeat :
      j := j - 1               # reduce exponent of phase kick
      CMul(v[j]) q[j], t       # perform a phase kick
      !jump_if_geq j, 0, @repeat

   invQFT[n] q                 # perform inverse QFT
   Mz q -> x                   # measure the result
   stop
```

The eigenvector $|\phi_1\rangle$ is not particularly helpful: it has eigenvalue $+1$, and so phase estimation will give a value of $x = 0$, which contains no information about t. However, we can generalise this idea to consider what happens if we have states with relative phases.

Another eigenvector is the state:

$$|\phi_{\mathrm{e}^{2\pi i/t}}\rangle = \tfrac{1}{\sqrt{t}}\Big(|1\rangle + \mathrm{e}^{2\pi i/t}|a\rangle + \mathrm{e}^{4\pi i/t}|a^2 \bmod N\rangle + \cdots + \mathrm{e}^{-2\pi i/t}|a^{t-1} \bmod N\rangle\Big). \quad (6.22)$$

We can easily see that again U_a shifts each of the terms of this sum, each of which will be off by a factor of $\mathrm{e}^{-2\pi i/t}$. Then, $|\phi_{\mathrm{e}^{2\pi i/t}}\rangle$ is a $\mathrm{e}^{-2\pi i/t}$-eigenvector of U_a. Similarly, we can consider a state:

$$|\phi_{\mathrm{e}^{4\pi i/t}}\rangle = \tfrac{1}{\sqrt{t}}\Big(|1\rangle \,+\, \mathrm{e}^{4\pi i/t}|a\rangle \,+\, \mathrm{e}^{8\pi i/t}|a^2 \bmod N\rangle \,+\cdots+\, \mathrm{e}^{-4\pi i/t}|a^{t-1} \bmod N\rangle\Big). \quad (6.23)$$

which we can show is a $\mathrm{e}^{-4\pi i/t}$-eigenvector of U_a, and so on. We can generalise these states to define eigenvectors:

$$|\phi_{\mathrm{e}^{2\pi ih/t}}\rangle = \tfrac{1}{\sqrt{t}}\Big(|1\rangle \,+\, \mathrm{e}^{2\pi ih/t}|a\rangle \,+\, \mathrm{e}^{4\pi ih/t}|a^2 \bmod N\rangle \,+\cdots+\, \mathrm{e}^{-2\pi ih/t}|a^{t-1} \bmod N\rangle\Big), \quad (6.24)$$

for all integers $0 \leqslant h < t$, where in each case, it is an $\mathrm{e}^{-2\pi ih/t}$-eigenvector of U_a.

We can't construct a single one of these states easily, without knowing what t is. However, we may consider whether we may effectively sample from these eigenstates, by considering computational basis states as superpositions of these eigenstates. For instance, notice that all of these states have the same contribution for the basis state $|1\rangle$. Because U_a is unitary, the eigenvectors $|\phi_{\mathrm{e}^{2\pi ih/t}}\rangle$ are orthogonal to each other. We might then want to consider the state $|\psi\rangle = \frac{1}{\sqrt{t}}(|\phi_1\rangle + |\phi_{\mathrm{e}^{2\pi ih/t}}\rangle + \cdots + |\phi_{\mathrm{e}^{-2\pi ih/t}}\rangle)$ – the uniform superposition of those eigenvectors – because the $|1\rangle$ components of each of these terms should add together. Indeed, we can show directly that:

$$\begin{aligned} |\psi\rangle \;&=\; \frac{1}{\sqrt{t}}\sum_{h=0}^{t-1}|\phi_{\mathrm{e}^{2\pi ih/t}}\rangle \;=\; \frac{1}{t}\sum_{h=0}^{t-1}\sum_{j=0}^{t-1}\mathrm{e}^{2\pi ijh/t}\,|a^j \bmod N\rangle \\ &=\; \frac{1}{t}\sum_{j=0}^{t-1}\left(\sum_{h=0}^{t-1}\left(\mathrm{e}^{2\pi ij/t}\right)^h\right)|a^j \bmod N\rangle\,. \end{aligned} \quad (6.25)$$

For $j \neq 0$, the sum in parentheses is a sum of roots of unity $\mathrm{e}^{2\pi ij/t}$ of the same sort that we contemplate in Figure 6.1 (though not necessarily 2^n-th roots of unity): then for $j \neq 0$, that sum is equal to 0. (For $j = 0$, the expression is instead a sum of t terms equal to 1, so that the sum evaluates to t.) Then we have:

$$|\psi\rangle \;=\; \frac{1}{t}\sum_{j=0}^{t-1}\left(\sum_{h=0}^{t-1}\left(\mathrm{e}^{2\pi ij/t}\right)^h\right)|a^j \bmod N\rangle \;=\; |1\rangle\,, \quad (6.26)$$

which is to say that:

$$|1\rangle \;=\; \tfrac{1}{\sqrt{t}}\Big(|\phi_1\rangle + |\phi_{\mathrm{e}^{2\pi i/t}}\rangle + |\phi_{\mathrm{e}^{4\pi i/t}}\rangle + \cdots + |\phi_{\mathrm{e}^{-2\pi i/t}}\rangle\Big). \quad (6.27)$$

Suppose then, that we use the m-qubit state $|1\rangle = |00\cdots01\rangle$ in place of an eigenvector $|\phi_{\mathrm{e}^{2\pi ih/t}}\rangle$. The phase-estimation procedure will proceed in a superposition of the different eigenvectors. When the control register is measured, an estimate of the eigenvalue of a uniformly random choice of state $|\phi_{\mathrm{e}^{2\pi ih/t}}\rangle$ will be the result. This will be a binary string $\tilde{x}$, providing (with high probability) an estimate of a **uniformly random** fraction

$x = h/t$, for $0 \leqslant h < t$.

This is more than enough to be able to quickly obtain enough information to estimate t, with high probability, with only a few attempts. If we take $n = 2m + 1 + \lceil \log_2(1/p) \rceil$, each random sample of $x = h/t$ contains enough information to reconstruct two integers (h', t') for which $h'/t' = h/t$, using a process known as the 'continued fractions algorithm' (see for example page 230 of Ref.[1]). If h has factors in common with t, then we may have $h'/t' = h/t$ but $t' \neq t$: but this does not matter; by taking multiple samples we can collect more samples to get more information about t. It is possible to show (see page 231 of Ref. [1]) that just two samples $x_1 = h_1/t$ and $x_2 = h_2/t$ are necessary to be able to compute t with probability at least $\frac{3}{4}$. Then we can find t in this way with probability of failure of at most $1/2^r$, using r samples $x_1 = h_1/t$, $x_2 = h_2/t$, ..., $x_r = h_r/t$. In each case, we may obtain this sample by initialising the m-qubit operand of `Phase_Estimation_Mul[n]` to $|1\rangle$.

6.3.6 Summary: Shor's Order Finding Algorithm

We can summarise the preceding section by describing how this may be realised in outline. (As the procedure that follows is almost entirely classical, with one step that uses a quantum subroutine, we write it out as a sequence of steps rather than pseudocode.) Note that the specific choices we describe to control the probability of failure and our description of the process of computing the order from different samples, differs somewhat from the original presentation in Ref, [3] and the presentation in Chapter 5 of Ref. [1].

In the following, fix some probability of failure p, and let $r = \lceil \log_2(p/2) \rceil$, and let $n = 2m + 1 + \lceil \log_2(2r/p) \rceil$.

1. Perform the following for $1 \leqslant j \leqslant r$ to obtain r different pairs of integers (h_1, t_1), (h_2, t_2), ..., (h_r, t_r):
    a. Initialise an m-qubit register `t` in the state $|00\cdots01\rangle$.
    b. Perform `Phase_Estimation_Mul[n] (a,N) t -> x`, and store the outcome as a fraction $0 < \tilde{x}_j < 1$ to n bits of precision.
    c. Apply the continued fractions algorithm to obtain an estimate $x_j = h_j \,/\, t_j$ for $\tilde{x}_j$, for m-bit integers h_j and t_j.
2. Compute the greatest common divisor of the fractions x_1, x_2, ..., x_r as follows. Let $j = 1$, and repeat the following until $j = r$, representing the various values x_j in reduced rational form (i.e., as a ratio $h_j \,/\, t_j$ for integers without common factors) at each computational step.
    a. If $x_j = 0$, increment j and return to this step.
    b. If $x_{j+1} = 0$, update $x_{j+1} := x_j$, increment j, and return to step (a).
    c. If $x_j < x_{j+1}$, then subtract from x_{j+1} the largest integer multiple of x_j possible without producing a negative number, and return to step (a).

d. Otherwise, then subtract from x_j the largest integer multiple of x_{j+1} possible without producing a negative number, and return to step (a).

3. Return the denominator t_r of the final value of x_r .

In Step 1, the probability of failure to obtain an accurate estimate of x_j (for some eigenvalue $e^{2\pi i x_j}$) to $2m$ bits, is $p/2r$ for each j: then the probability with which **at least one** of the estimates of x_j is inaccurate to $2m$ bits is at most $p/2$. In Step 2, the probability with which this process fails to actually produce the period of a, given that Step 1 succeeded, is at most $2^r < p/2$. Then, this procedure produces the period of a, with a probability of failure of at most p.

Summary

This week's Study material, is by far the most technical of the content we have encountered so far. You are not expected to remember it in fine detail: the most important results are that:

- integer factorisation, is a problem that is difficult enough that it has been used to underpin important cryptographic systems
- using various techniques from number theory, it is possible to reduce this problem to the 'order finding' problem, of determining the smallest exponent $t > 1$ such that $a^t \equiv 1 \pmod{N}$, for a unit a modulo N
- finding the order of a modulo N is closely tied to the question of what the eigenvalues are, of the unitary transformation U_a such that $U_a |x\rangle = |ax \bmod N\rangle$ for $0 \leqslant x < N$
- through considering computational basis states as superpositions of eigenstates, we can find the order of a mod N through sampling the eigenvalues of random eigenvectors of U_a
- finally, this is in the end a hybrid quantum/classical algorithm, with a probability of failure which can be bounded by making an appropriate number of attempts for certain steps, and by providing an appropriate amount of space to obtain estimates of the phase of eigenvectors.

References

[1] Nielsen, M. A. and Chuang, I. L. Quantum Computation and Quantum Information. 10th anniversary edn. Cambridge, UK: Cambridge University Press, 2010.

[2] Bach, E. and Shallit, J. O. Algorithmic Number Theory. Vol I: Efficient Algorithms. Cambridge, MA: MIT Press, 1996.

[3] Shor, P. W. Algorithms for quantum computation: Discrete logarithms and factoring. Proceedings 35th Annual Symposium on Foundations of Computer Science. IEEE Computer Society Press, 1994. [Online], pp. 124–134. doi:10.1109/sfcs.1994.365700 [accessed 7 September 2023].